

Guia de Implementação — Tarefa 1: O Início da Operação

Linha de Produção Industrial em Java com POO

Estrutura do Projeto

```
tarefa1/  
├── src/  
│   ├── MateriaPrima.java  
│   ├── Produto.java  
│   ├── Maquina.java  
│   ├── Esteira.java  
│   ├── EstacaoInspecao.java  
│   └── Main.java
```

Cada classe fica em seu próprio arquivo `.java`. Isso respeita o princípio de **Single Responsibility** (cada arquivo = uma responsabilidade).

Conceito-chave antes de começar: Encapsulamento

Todas as classes seguem o mesmo padrão:

```
atributos → sempre PRIVATE  
métodos   → sempre PUBLIC (getters, setters, comportamentos)
```

Isso garante que **nenhuma classe modifique diretamente o estado interno de outra**. O estado de uma `Maquina`, por exemplo, só muda pelos métodos `ligar()` e `desligar()` — nunca por código externo acessando `maquina.ligada = true` diretamente.

Classe 1: `MateriaPrima.java`

Responsabilidade: gerenciar o estoque de matéria-prima.

```

public class MateriaPrima {

    // Atributos PRIVADOS – encapsulamento
    private int id;
    private String nome;
    private double quantidade;
    private String unidade;
    private double quantidadeMinima;

    // Construtor: define os valores iniciais no momento da criação do objeto
    public MateriaPrima(int id, String nome, double quantidade,
                        String unidade, double quantidadeMinima) {
        this.id = id;
        this.nome = nome;
        this.quantidade = quantidade;
        this.unidade = unidade;
        this.quantidadeMinima = quantidadeMinima;
    }

    // Reduz a quantidade disponível conforme a demanda consumida
    public void consumir(double demanda) {
        if (verificarDisponibilidade(demanda)) {
            this.quantidade -= demanda;
            System.out.println("Consumidos " + demanda + " " + unidade
                              + " de " + nome + ". Restante: " + quantidade);
        } else {
            System.out.println("[ERRO] Estoque insuficiente para consumir " + demanda + " " + unidade);
        }
    }

    // Aumenta o estoque disponível
    public void adicionarEstoque(double qtd) {
        this.quantidade += qtd;
        System.out.println("Adicionados " + qtd + " " + unidade
                          + " ao estoque de " + nome + ". Total: " + quantidade);
    }

    // Verifica se há quantidade suficiente para a demanda informada
    public boolean verificarDisponibilidade(double demanda) {
        return this.quantidade >= demanda;
    }

    // Getters – única forma de acessar os atributos privados
    public int getId() { return id; }
    public String getNome() { return nome; }
    public double getQuantidade() { return quantidade; }
    public String getUnidade() { return unidade; }
    public double getQuantidadeMinima() { return quantidadeMinima; }
}

```

Conceito em destaque: `verificarDisponibilidade()` é chamada tanto internamente (dentro de `consumir()`) quanto externamente (pelo `Main` antes de iniciar a produção). Isso aplica o princípio **DRY** — a lógica de verificação existe em um único lugar.

Classe 2: `Produto.java`

Responsabilidade: representar o produto e controlar seu ciclo de vida na linha.

```

public class Produto {

    // Constantes de status — KISS: nomes claros evitam Strings mágicas espalhadas
    public static final String STATUS_AGUARDANDO = "AGUARDANDO_PROCESSAMENTO";
    public static final String STATUS_PROCESSADO = "PROCESSADO";
    public static final String STATUS_INSPECIONADO = "INSPECIONADO";

    // Atributos PRIVADOS
    private int id;
    private String nome;
    private String status;
    private double quantidadeMateriaPrimaNecessaria;

    // Construtor: produto começa sempre aguardando processamento
    public Produto(int id, String nome, double quantidadeMateriaPrimaNecessaria) {
        this.id = id;
        this.nome = nome;
        this.status = STATUS_AGUARDANDO;
        this.quantidadeMateriaPrimaNecessaria = quantidadeMateriaPrimaNecessaria;
    }

    // Avança o status do produto para PROCESSADO
    public void processar() {
        this.status = STATUS_PROCESSADO;
        System.out.println("Produto '" + nome + "' marcado como: " + status);
    }

    // Avança o status do produto para INSPECIONADO
    public void inspecionar() {
        this.status = STATUS_INSPECIONADO;
        System.out.println("Produto '" + nome + "' marcado como: " + status);
    }

    // Permite redefinir a demanda de matéria-prima em tempo de execução
    public void definirDemandaMateriaPrima(double qtd) {
        this.quantidadeMateriaPrimaNecessaria = qtd;
    }

    // Getters
    public double getDemandaMateriaPrima() { return quantidadeMateriaPrimaNecessaria; }
    public int getId() { return id; }
    public String getNome() { return nome; }
    public String getStatus(){ return status; }
}

```

Conceito em destaque: O status jamais é alterado diretamente — apenas pelos métodos `processar()` e `inspecionar()`. Isso é **encapsulamento de comportamento**: o objeto controla sua própria mudança de estado.

Classe 3: `Maquina.java`

Responsabilidade: transformar matéria-prima em produto, respeitando seu estado (ligada/desligada).

```

public class Maquina {

    // Atributos PRIVADOS
    private String nome;
    private boolean ligada;
    private double capacidadeMaxima;

    // Construtor: máquina começa desligada
    public Maquina(String nome, double capacidadeMaxima) {
        this.nome = nome;
        this.ligada = false;
        this.capacidadeMaxima = capacidadeMaxima;
    }

    public void ligar() {
        this.ligada = true;
        System.out.println("Máquina '" + nome + "' LIGADA.");
    }

    public void desligar() {
        this.ligada = false;
        System.out.println("Máquina '" + nome + "' DESLIGADA.");
    }

    // Processa: consome MP e avança o status do produto
    // Retorna true se o processamento foi bem-sucedido
    public boolean processar(MateriaPrima mp, double demanda, Produto produto) {
        if (!ligada) {
            System.out.println("[ERRO] Máquina '" + nome + "' está desligada. Ligue-a primeiro.");
            return false;
        }
        if (demanda > capacidadeMaxima) {
            System.out.println("[ERRO] Demanda (" + demanda + ") excede capacidade máxima (" + capacidadeMaxima + ").");
            return false;
        }
        if (!mp.verificarDisponibilidade(demanda)) {
            System.out.println("[ERRO] Matéria-prima insuficiente para processar '" + produto.getNome() + "'.");
            return false;
        }

        mp.consumir(demanda);
        produto.processar();
        System.out.println(">>> Processamento concluído: '" + produto.getNome() + "' fabricado com sucesso!");
        return true;
    }

    // Getters
    public String getNome() { return nome; }
    public boolean estaLigada() { return ligada; }
    public double getCapacidadeMaxima() { return capacidadeMaxima; }
}

```

Conceito em destaque: `processar()` recebe objetos de outras classes como parâmetro — isso é **composição**: a `Maquina` usa `MateriaPrima` e `Produto`, mas não "herda" deles nem "possui" permanentemente. A dependência é pontual, via parâmetro.

Classe 4: Esteira.java

Responsabilidade: transportar itens (MP ou Produto) entre os estágios da linha.

```

public class Esteira {

    // 'Object' permite transportar qualquer tipo (MP ou Produto)
    // sem usar herança – estratégia adequada para a Tarefa 1
    private Object item;
    private boolean emMovimento;
    private double capacidadeMaxima;

    // Construtor: esteira começa parada e sem item
    public Esteira(double capacidadeMaxima) {
        this.item = null;
        this.emMovimento = false;
        this.capacidadeMaxima = capacidadeMaxima;
    }

    public void ligar() {
        this.emMovimento = true;
        System.out.println("Esteira em MOVIMENTO.");
    }

    public void desligar() {
        this.emMovimento = false;
        System.out.println("Esteira PARADA.");
    }

    // Coloca um item na esteira – falha se já tiver item ou estiver parada
    public boolean adicionarItem(Object novoItem) {
        if (!emMovimento) {
            System.out.println("[ERRO] Esteira parada. Ligue-a antes de adicionar itens.");
            return false;
        }
        if (this.item != null) {
            System.out.println("[ERRO] Esteira já contém um item. Remova-o antes.");
            return false;
        }
        this.item = novoItem;
        System.out.println("Item adicionado à esteira e sendo transportado...");
        return true;
    }

    // Remove e retorna o item transportado
    public Object removerItem() {
        if (!emMovimento) {
            System.out.println("[ERRO] Esteira parada. Não é possível remover itens.");
            return null;
        }
        if (this.item == null) {
            System.out.println("[AVISO] Esteira está vazia.");
            return null;
        }
        Object itemRemovido = this.item;
        this.item = null;
        System.out.println("Item removido da esteira.");
        return itemRemovido;
    }

    // Verifica se a esteira suporta o peso/volume do item
    public boolean verificarCapacidade(double peso) {
        return peso <= capacidadeMaxima;
    }
}

```

```
// Getters
public boolean estaEmMovimento() { return emMovimento; }
public boolean temItem() { return item != null; }
public double getCapacidadeMaxima() { return capacidadeMaxima; }
}
```

Conceito em destaque: A `Esteira` não sabe o que está transportando — só sabe que tem um `Object`. Quando quem recebe o item precisa usá-lo como `MateriaPrima` ou `Produto`, faz um **cast** explícito: `(MateriaPrima) esteira.removerItem()`. Em Tarefa 2, isso será resolvido com herança/interfaces.

Classe 5: `EstacaoInspecao.java`

Responsabilidade: inspecionar o produto e aprovar sua qualidade.

```
public class EstacaoInspecao {

    // Atributos PRIVADOS
    private boolean ativa;
    private int produtosInspeccionados;

    // Construtor: estação começa desativada
    public EstacaoInspecao() {
        this.ativa = false;
        this.produtosInspeccionados = 0;
    }

    public void ativar() {
        this.ativa = true;
        System.out.println("Estação de inspeção ATIVADA.");
    }

    public void desativar() {
        this.ativa = false;
        System.out.println("Estação de inspeção DESATIVADA.");
    }

    // Inspecciona o produto — falha se estação estiver desativada
    public boolean inspecionar(Produto produto) {
        if (!ativa) {
            System.out.println("[ERRO] Estação de inspeção está desativada.");
            return false;
        }
        produto.inspecionar(); // delega ao próprio Produto a mudança de status
        produtosInspeccionados++;
        System.out.println(">>> Inspeção concluída: produto APROVADO! Total inspeccionado: " + produtosInspeccionados);
        return true;
    }

    // Getter
    public int getTotalInspeccionados() { return produtosInspeccionados; }
    public boolean estaAtiva() { return ativa; }
}
```

Conceito em destaque: A `EstacaoInspecao` **delega** a mudança de status ao próprio `Produto` via `produto.inspecionar()`. Ela não altera diretamente o estado interno do produto — respeita o encapsulamento do objeto `Produto`.

Classe 6: `Main.java`

Responsabilidade: orquestrar a linha de produção via menu de terminal.


```

import java.util.Scanner;

public class Main {

    // Scanner estático – compartilhado entre os métodos (DRY: um único Scanner)
    private static Scanner scanner = new Scanner(System.in);

    public static void main(String[] args) {

        // — 1. Instanciar Matéria-Prima —————
        MateriaPrima aco = new MateriaPrima(1, "Aço", 200.0, "kg", 10.0);

        // — 2. Instanciar Produtos (definidos no código, não pelo usuário) —
        Produto[] produtos = {
            new Produto(1, "Parafuso", 5.0),
            new Produto(2, "Engrenagem", 20.0),
            new Produto(3, "Porca", 3.0)
        };

        // — 3. Instanciar Máquina —————
        Maquina maquina = new Maquina("Prensa Industrial", 500.0);

        // — 4. Instanciar Esteira —————
        Esteira esteira = new Esteira(1000.0);

        // — 5. Instanciar Estação de Inspeção —————
        EstacaoInspecao estacao = new EstacaoInspecao();

        // — 6. Loop do Menu —————
        int opcao = -1;
        while (opcao != 0) {
            exibirMenu();
            opcao = lerInteiro("Escolha uma opção: ");

            switch (opcao) {
                case 1:
                    iniciarProducao(aco, produtos, maquina, esteira, estacao);
                    break;
                case 2:
                    consultarEstoque(aco);
                    break;
                case 3:
                    adicionarEstoque(aco);
                    break;
                case 0:
                    System.out.println("\nSistema encerrado.");
                    System.out.println("Total de produtos inspecionados: " + estacao.getTotalInspeccionados());
                    break;
                default:
                    System.out.println("[AVISO] Opção inválida. Digite apenas os números do menu.");
            }
        }
        scanner.close();
    }

    // — Método: exibir menu —————
    private static void exibirMenu() {
        System.out.println("\n=====");
        System.out.println("|| SISTEMA DE PRODUÇÃO INDUSTRIAL ||");
    }
}

```

```

        System.out.println("||=====||");
        System.out.println("|| 1. Iniciar produção ||");
        System.out.println("|| 2. Consultar estoque de MP ||");
        System.out.println("|| 3. Adicionar estoque de MP ||");
        System.out.println("|| 0. Sair ||");
        System.out.println("||=====||");
    }

    // — Método: fluxo completo de produção —————
    private static void iniciarProducao(MateriaPrima mp, Produto[] produtos,
                                         Máquina maquina, Esteira esteira,
                                         EstacaoInspecao estacao) {
        System.out.println("\n--- SELEÇÃO DE PRODUTO ---");
        for (int i = 0; i < produtos.length; i++) {
            System.out.printf(" %d. %-15s (demanda padrão: %.1f %s)%n",
                              i + 1, produtos[i].getNome(),
                              produtos[i].getDemandaMateriaPrima(), mp.getUnidade());
        }

        int escolha = lerInteiro("Selecione o produto (1-" + produtos.length + "): ");
        if (escolha < 1 || escolha > produtos.length) {
            System.out.println("[ERRO] Produto inválido.");
            return;
        }

        Produto produtoSelecioneado = produtos[escolha - 1];

        // Reinicia status do produto para nova produção
        // (necessário se o produto já foi produzido antes)
        System.out.println("Produto selecionado: " + produtoSelecioneado.getNome()
                           + " | Status atual: " + produtoSelecioneado.getStatus());

        int demanda = lerInteiro("Informe a quantidade de matéria-prima a utilizar (" + mp.getUnidade() + "): ");

        // — 9. Verificar disponibilidade —————
        if (!mp.verificarDisponibilidade(demanda)) {
            System.out.printf("[ERRO] Estoque insuficiente! Disponível: %.1f %s | Solicitado: %d %s%n",
                              mp.getQuantidade(), mp.getUnidade(), demanda, mp.getUnidade());
            return;
        }

        System.out.println("\n>>> Iniciando fluxo de produção...");

        // — 10. Ligar equipamentos —————
        maquina.ligar();
        esteira.ligar();
        estacao.ativar();

        // — 11-12. Colocar MP na esteira e transportar —————
        esteira.adicionarItem(mp);
        System.out.println("Transportando matéria-prima para a máquina...");
        MateriaPrima mpTransportada = (MateriaPrima) esteira.removerItem();

        // — 13. Processar na máquina —————
        boolean processado = maquina.processar(mpTransportada, demanda, produtoSelecioneado);
        if (!processado) {
            desligarEquipamentos(maquina, esteira, estacao);
            return;
        }
    }

```

```

// — 14. Transportar produto para inspeção —————
esteira.adicionarItem(produtoSelecionado);
System.out.println("Transportando produto para estação de inspeção...");
Produto produtoTransportado = (Produto) esteira.removerItem();

// — 15. Inspecionar —————
estacao.inspecionar(produtoTransportado);

// — 17. Informar status final —————
System.out.println("\n=====");
System.out.println("||          RESUMO DA PRODUÇÃO          ||");
System.out.println("||=====||");
System.out.printf( "|| Produto:  %-21s||\n", produtoTransportado.getNome());
System.out.printf( "|| Status:   %-21s||\n", produtoTransportado.getStatus());
System.out.printf( "|| MP usada:  %-17d %s  ||\n", demanda, mp.getUnidade());
System.out.printf( "|| MP restante: %.1f %-12s||\n", mp.getQuantidade(), mp.getUnidade());
System.out.printf( "|| Total inspec.: %-17d||\n", estacao.getTotalInspecionados());
System.out.println("||=====||");

desligarEquipamentos(maquina, esteira, estacao);
}

// — Método: consultar estoque —————
private static void consultarEstoque(MateriaPrima mp) {
    System.out.println("\n--- ESTOQUE DE MATÉRIA-PRIMA ---");
    System.out.println("Nome:      " + mp.getNome());
    System.out.printf( "Disponível: %.1f %s\n", mp.getQuantidade(), mp.getUnidade());
    System.out.printf( "Mínimo:      %.1f %s\n", mp.getQuantidadeMinima(), mp.getUnidade());

    if (mp.getQuantidade() < mp.getQuantidadeMinima()) {
        System.out.println("[ALERTA] Estoque abaixo do nível mínimo!");
    }
}

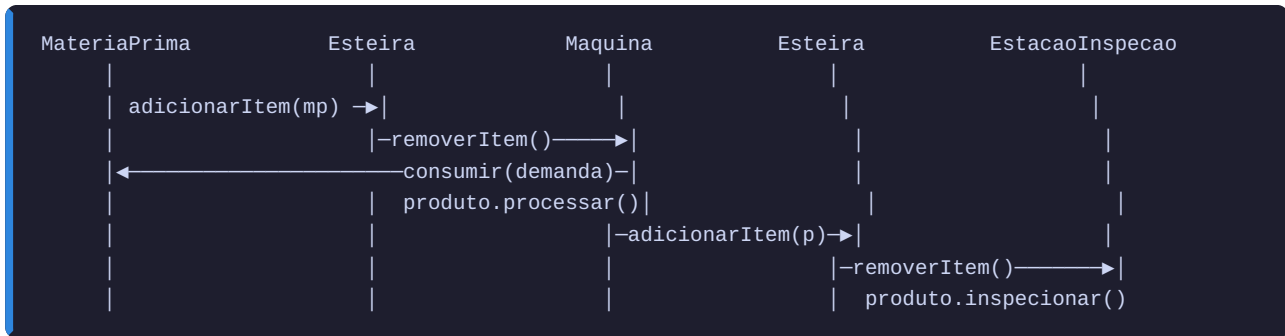
// — Método: adicionar estoque —————
private static void adicionarEstoque(MateriaPrima mp) {
    int qtd = lerInteiro("Quantidade a adicionar (" + mp.getUnidade() + "): ");
    mp.adicionarEstoque(qtd);
}

// — Método: desligar todos os equipamentos —————
private static void desligarEquipamentos(Maquina maquina, Esteira esteira,
                                          EstacaoInspecao estacao) {
    System.out.println("\nDesligando equipamentos...");
    maquina.desligar();
    esteira.desligar();
    estacao.desativar();
}

// — DRY: leitura segura de inteiro —————
// Princípio DRY: lógica de validação em um único lugar
private static int lerInteiro(String mensagem) {
    System.out.print(mensagem);
    while (!scanner.hasNextInt()) {
        System.out.print("[ERRO] Digite apenas números inteiros: ");
        scanner.next(); // descarta entrada inválida
    }
    int valor = scanner.nextInt();
    scanner.nextLine(); // limpa o buffer
    return valor;
}
}

```

Diagrama do Fluxo de Produção



Resumo dos Conceitos Aplicados

Conceito	Onde aparece no código
Encapsulamento	Todos os atributos <code>private</code> + getters públicos em todas as classes
Composição	<code>Maquina.processar()</code> recebe <code>MateriaPrima</code> e <code>Produto</code> como parâmetros
Controle de estado	<code>ligada</code> , <code>emMovimento</code> , <code>ativa</code> controlados por <code>ligar()</code> / <code>desligar()</code>
Single Responsibility	Cada classe tem uma única função na linha de produção
DRY	<code>lerInteiro()</code> centraliza a lógica de leitura numérica; <code>verificarDisponibilidade()</code> usada em dois lugares
KISS	Sem herança, sem banco de dados, sem GUI — só o que a tarefa pede
YAGNI	Não foi implementado nada além do que está no enunciado

Guia gerado com base na análise do PDF "Tarefa 1 — O Início da Operação".

Mapa de Estudo: Código → Conceito → Livro → Capítulo

Tarefa 1 — O Início da Operação Estratégia: Leia o capítulo indicado → estude o código correspondente → reescreva com suas próprias palavras.

Como usar este mapa

- Escolha uma classe para começar (recomendado: `MateriaPrima`).
- Abra o livro indicado no capítulo correspondente.
- Leia o capítulo inteiro (ou as seções indicadas).
- Volte ao código da classe e identifique onde cada conceito aparece.
- Feche o guia e tente reescrever a classe do zero.

6. Compare com o original e corrija o que for diferente.
 7. Passe para a próxima classe.
-

Tabela Principal: Classe → Conceito → Livro → Capítulo

Classe / Trecho de Código	Conceito POO Aplicado	Livro Principal	Capítulo / Seção
MateriaPrima	Classe, atributos, construtor	<i>Objects First with Java</i> — Barnes & Kölling	Cap. 2: Defining Classes
<code>private String nome</code>	Encapsulamento (atributo privado)	<i>Objects First with Java</i>	Cap. 2: Fields and Encapsulation
<code>public MateriaPrima(String nome, int quantidade)</code>	Construtor com parâmetros	<i>Objects First with Java</i>	Cap. 2: Constructors
<code>getNome(), getQuantidade()</code>	Métodos getters (acesso controlado)	<i>Objects First with Java</i>	Cap. 2: Accessor Methods
<code>setQuantidade(int q)</code>	Método setter (mutador)	<i>Objects First with Java</i>	Cap. 2: Mutator Methods
<code>toString()</code>	Sobrescrita de método herdado de Object	<i>Objects First with Java</i>	Cap. 9: The toString Method
Produto	Classe com múltiplos atributos e status	<i>Objects First with Java</i> — Barnes & Kölling	Cap. 2 e Cap. 3
<code>static final String AGUARDANDO...</code>	Constantes estáticas (evita strings mágicas)	<i>Building Maintainable Software</i> — Visser et al.	Guideline 4: Write Code Once (DRY)
<code>private String status</code>	Estado interno do objeto	<i>Objects First with Java</i>	Cap. 3: Object State
<code>inspecionar()</code>	Responsabilidade única por classe (SRP)	<i>Building Maintainable Software</i>	Guideline 1: Write Short Units of Code
<code>toString()</code> em Produto	Representação textual do objeto	<i>Objects First with Java</i>	Cap. 9: The toString Method
Maquina	Classe com comportamento (método processar)	<i>Introduction to Programming with Java</i> — Dean & Dean	Cap. 4: Using Objects
<code>processar(Produto p)</code>	Passagem de objeto como parâmetro	<i>Introduction to Programming with Java</i>	Cap. 7: Object-Oriented Programming
Alteração do status dentro de <code>processar()</code>	Interação entre objetos	<i>Objects First with Java</i>	Cap. 3: Object Interaction
<code>toString()</code> em Maquina	Representação textual	<i>Objects First with Java</i>	Cap. 9: The toString Method
Esteira	Estrutura de dados: array de objetos	<i>Introduction to Programming with Java</i> — Dean & Dean	Cap. 7: Arrays of Objects
<code>private Object[] itens</code>	Array genérico (sem herança)	<i>Introduction to Programming with Java</i>	Cap. 7: Array Basics
<code>adicionarItem(Object item)</code>	Método com lógica condicional	<i>Introduction to Programming with Java</i>	Cap. 3: If Statements
<code>removerItem()</code>	Retorno de objeto, verificação de nulo	<i>Introduction to Programming with Java</i>	Cap. 4: Return Values

Classe / Trecho de Código	Conceito POO Aplicado	Livro Principal	Capítulo / Seção
<code>estaCheia()</code> , <code>estaVazia()</code>	Métodos booleanos de consulta de estado	<i>Objects First with Java</i>	Cap. 2: Methods with Return Values
EstacaoInspecao	Delegação de responsabilidade entre classes	<i>Objects First with Java</i> — Barnes & Kölling	Cap. 3: Collaboration Between Classes
<code>inspecionar(Produto p)</code>	Método que chama método de outro objeto	<i>Objects First with Java</i>	Cap. 3: Object Interaction
Verificação de <code>status</code> antes de agir	Lógica condicional com estado de objeto	<i>Introduction to Programming with Java</i>	Cap. 3: Conditional Execution
Main (classe principal)	Ponto de entrada do programa, Scanner	<i>Introduction to Programming with Java</i> — Dean & Dean	Cap. 3: User Input with Scanner
<code>Scanner sc = new Scanner(System.in)</code>	Leitura de entrada numérica do usuário	<i>Introduction to Programming with Java</i>	Cap. 3: The Scanner Class
<code>switch</code> ou <code>if-else</code> para menu	Estrutura de controle de fluxo	<i>Introduction to Programming with Java</i>	Cap. 3: Switch Statement
<code>while (opcao != 0)</code>	Laço de repetição (loop de menu)	<i>Introduction to Programming with Java</i>	Cap. 3: While Loops
Instanciação de objetos em Main	Uso de construtores e referências	<i>Objects First with Java</i>	Cap. 1: Creating Objects
Chamada de métodos em Main	Invocação de comportamento	<i>Objects First with Java</i>	Cap. 1: Calling Methods

Ordem de Leitura Recomendada

Siga esta sequência para maximizar o aprendizado com pouco tempo disponível:

Etapas	O que ler	Onde ler	O que implementar em seguida
1	Objetos e classes básicas	<i>Objects First</i> — Cap. 1 e Cap. 2	<code>MateriaPrima</code> e <code>Produto</code> (atributos + construtor + getters)
2	Interação entre objetos	<i>Objects First</i> — Cap. 3	<code>Maquina</code> e <code>EstacaoInspecao</code>
3	Arrays de objetos	<i>Dean & Dean</i> — Cap. 7	<code>Esteira</code> (array + adicionar + remover)
4	Entrada do usuário e loops	<i>Dean & Dean</i> — Cap. 3	<code>Main</code> (Scanner + menu + laço)
5	Boas práticas e DRY	<i>Building Maintainable Software</i> — Guideline 4	Revisar constantes estáticas e eliminar repetição
6	<code>toString</code> e representação	<i>Objects First</i> — Cap. 9	Adicionar/revisar <code>toString()</code> em todas as classes

Resumo dos Livros Usados

Livro	Autores	Foco Principal nesta Tarefa
<i>Objects First with Java: A Practical Introduction Using BlueJ</i> (6ª ed.)	David J. Barnes, Michael Kölling	Classes, encapsulamento, interação entre objetos, toString
<i>Introduction to Programming with Java: A Problem Solving Approach</i> (2ª ed.)	John Dean, Raymond Dean	Scanner, arrays, loops, estruturas de controle
<i>Building Maintainable Software, Java Edition</i>	Joost Visser et al.	DRY (Write Code Once), métodos curtos, responsabilidade única

Dica Final

Nunca copie o código diretamente. Use o guia para entender a lógica, leia o capítulo do livro, e depois escreva cada classe com suas próprias palavras. Isso garante aprendizado real e integridade acadêmica.

Documento gerado como material de apoio para a Tarefa 1 — O Início da Operação.